

```

import math

import numpy as np

# =====

# QUESTION 1: POINT CLASS

# =====

class Point:

    def __init__(self, x, y):

        self.x = x

        self.y = y

    def distance(self, other):

        return math.sqrt((self.x - other.x)**2 + (self.y - other.y)**2)

    def midpoint(self, other):

        return Point((self.x + other.x)/2, (self.y + other.y)/2)

# Reflection of point C over line AB

def reflect_point(A, B, C):

    dx = B.x - A.x

    dy = B.y - A.y

    a = dy

    b = -dx

    c = dx*A.y - dy*A.x

    d = (a*C.x + b*C.y - c) / (a*a + b*b)

    x_ref = C.x - 2*a*d

    y_ref = C.y - 2*b*d

    return Point(x_ref, y_ref)

```

```

# =====

# QUESTION 2: VECTORS

# =====

def magnitude(v):
    return np.linalg.norm(v)

def angle_between(v1, v2):
    cos_theta = np.dot(v1, v2) / (magnitude(v1) * magnitude(v2))
    return math.degrees(math.acos(cos_theta))

def projection(a, b):
    return (np.dot(a, b) / np.dot(b, b)) * b

# =====

# QUESTION 3: SEGMENT

# =====

def closest_point_on_segment(S, E, P):
    SE = np.array([E.x - S.x, E.y - S.y])
    SP = np.array([P.x - S.x, P.y - S.y])

    t = np.dot(SP, SE) / np.dot(SE, SE)
    t = max(0, min(1, t))

    closest = Point(S.x + t*SE[0], S.y + t*SE[1])
    return closest

# =====

# QUESTION 4: LINE INTERSECTION

# =====

def line_intersection(a1, b1, c1, a2, b2, c2):

```

```

det = a1*b2 - a2*b1

if det == 0:

    return None

x = (c1*b2 - c2*b1) / det

y = (a1*c2 - a2*c1) / det

return x, y


# =====

# MAIN EXECUTION

# =====

print("\n--- QUESTION 1: POINT OPERATIONS ---")

A = Point(2, 3)

B = Point(6, 7)

C = Point(4, 2)


print("Distance AB:", A.distance(B))

mid = A.midpoint(B)

print("Midpoint AB:", (mid.x, mid.y))


m = (B.y - A.y) / (B.x - A.x)

c = A.y - m*A.x

print(f"Line Equation: y = {m}x + {c}")


ref = reflect_point(A, B, C)

print("Reflection of C:", (ref.x, ref.y))


print("\n--- QUESTION 2: VECTOR OPERATIONS ---")

A_vec = np.array([2, 3])

B_vec = np.array([4, 1])

C_vec = np.array([1, 5])

```

```
R = A_vec + B_vec + C_vec
```

```
print("Resultant Vector:", R)
```

```
print("Magnitudes:", magnitude(A_vec), magnitude(B_vec), magnitude(C_vec))
```

```
print("Dot Products:")
```

```
print("A·B:", np.dot(A_vec, B_vec))
```

```
print("A·C:", np.dot(A_vec, C_vec))
```

```
print("B·C:", np.dot(B_vec, C_vec))
```

```
print("Angles (degrees):")
```

```
print("A & B:", angle_between(A_vec, B_vec))
```

```
print("A & C:", angle_between(A_vec, C_vec))
```

```
print("B & C:", angle_between(B_vec, C_vec))
```

```
proj = projection(A_vec, B_vec)
```

```
print("Projection of A on B:", proj)
```

```
print("\n--- QUESTION 3: SEGMENT OPERATIONS ---")
```

```
S = Point(1, 2)
```

```
E = Point(6, 5)
```

```
P = Point(4, 1)
```

```
seg_len = S.distance(E)
```

```
print("Segment Length:", seg_len)
```

```
closest = closest_point_on_segment(S, E, P)
```

```
print("Closest Point:", (closest.x, closest.y))
```

```
dist = P.distance(closest)
```

```
print("Distance from P to Segment:", dist)
```

```
print("\n--- QUESTION 4: LINE INTERSECTION ---")
```

```
a1, b1, c1 = 2, -1, 3
```

```
a2, b2, c2 = 1, 1, 5
```

```
inter = line_intersection(a1, b1, c1, a2, b2, c2)
```

```
if inter:
```

```
    print("Intersection Point:", inter)
```

```
else:
```

```
    print("Lines are parallel or coincident.")
```